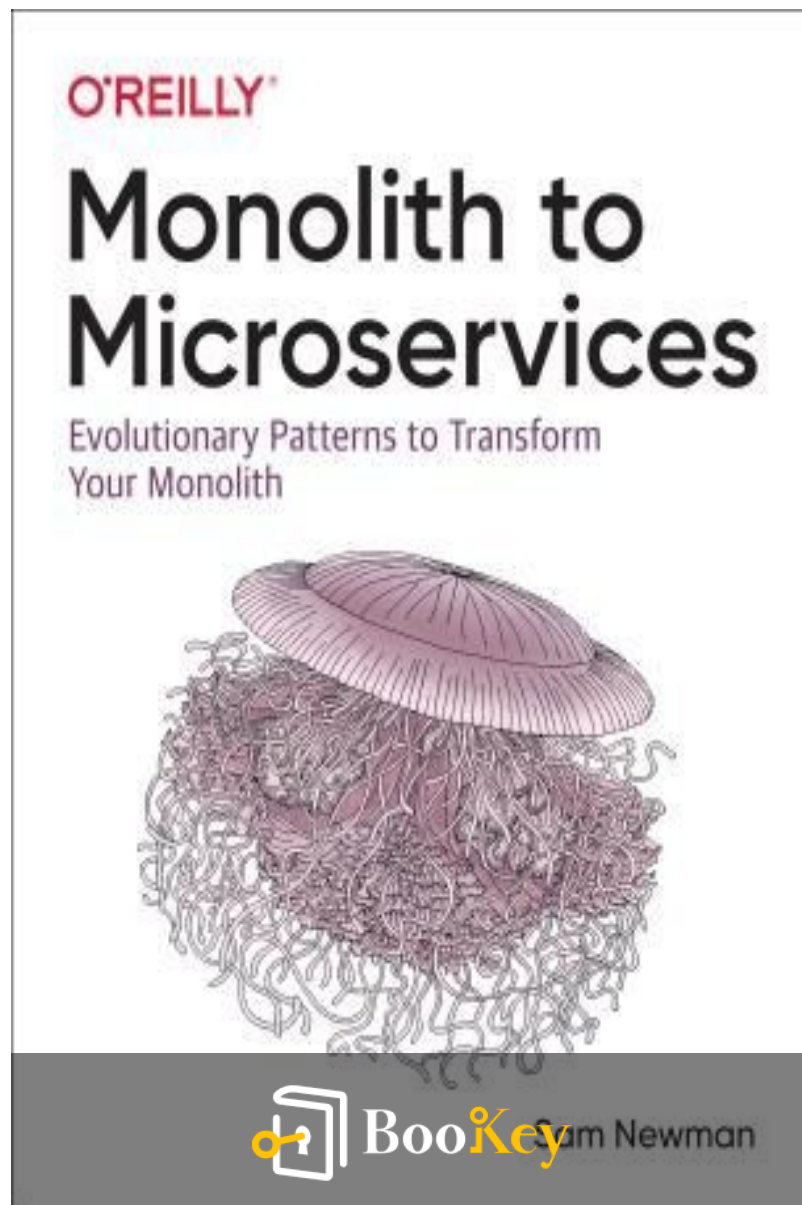


Monolith to Microservices PDF

Sam Newman



More Free Book



Scan to Download



Listen It

Monolith to Microservices

Practical Strategies for Transitioning from Monolith
to Microservices.

Written by Bookey

[Check more about Monolith to Microservices Summary](#)

[Listen Monolith to Microservices Audiobook](#)

More Free Book



Scan to Download



[Listen It](#)

About the book

In "Monolith to Microservices," Sam Newman presents a practical guide for organizations seeking to transition from a monolithic architecture to a microservices framework without disrupting ongoing operations. Building on the foundational concepts from his highly regarded "Building Microservices," this book offers a comprehensive exploration of proven strategies for effective migration. With a wealth of illustrative examples, insightful migration patterns, and actionable advice, readers will learn how to navigate the complexities of application and database decomposition. The guide addresses critical considerations for planning, communication, and integration as well as provides practical techniques for dealing with legacy systems. Whether assessing readiness for migration or tackling specific challenges, this resource is invaluable for those committed to enhancing their architecture while maintaining business continuity.

More Free Book



Scan to Download



Listen It

About the author

Sam Newman is a prominent thought leader and consultant in the field of software architecture, widely recognized for his expertise in microservices and cloud-native technologies. With a background in software development and a passion for helping organizations navigate the complexities of modern software systems, Newman has become a sought-after speaker and educator, sharing valuable insights on design principles, team dynamics, and DevOps practices. His influential book, "Monolith to Microservices," draws on his extensive experience in helping companies transition to more scalable and agile architectures, making a significant impact on the way organizations approach software design and delivery in the face of evolving business demands.

More Free Book



Scan to Download



Listen It

Ad



Scan to Download



Try Bookey App to read 1000+ summary of world best books

Unlock **1000+** Titles, **80+** Topics

New titles added every week

Brand



Leadership & Collaboration



Time Management



Relationship & Communication



Business Strategy



Creativity



Public



Money & Investing



Know Yourself



Positive Psychology

Entrepreneurship



World History



Parent-Child Communication

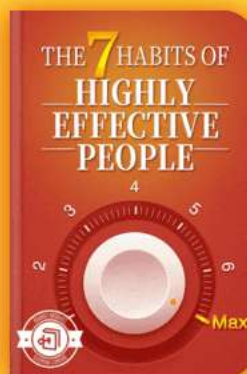
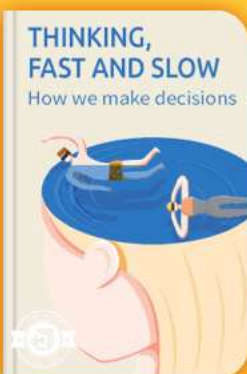


Self-care



Mind & Spirituality

Insights of world best books



Free Trial with Bookey



Summary Content List

Chapter 1 :

Chapter 2 :

Chapter 3 :

Chapter 4 :

Chapter 5 :

Chapter 6 :

Chapter 7 :

More Free Book

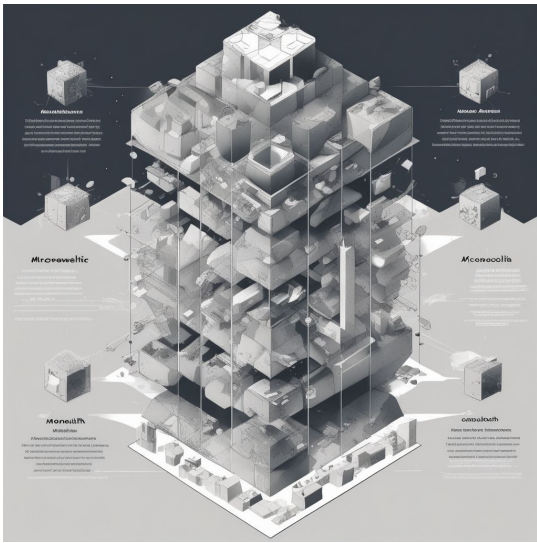


Scan to Download



Listen It

Chapter 1 Summary :



Summary of Chapter 1: Monolith to Microservices

Change Scope and Multi-Tier Architecture

- Implementing a new feature, like allowing customers to choose a music genre, requires a coordinated effort across multiple tiers: Web UI, Backend, and Database.
- Common three-tier architecture aligns with traditional team organization but lacks flexibility for rapid changes in business functionality.

Conway's Law and Team Organization

More Free Book



Scan to Download



Listen It

- Conway's Law suggests that system design reflects organizational communication structures, leading to tightly coupled technology-driven systems rather than cohesive business domains.
- Modern practices focus on poly-skilled teams to improve collaboration and speed up software delivery.

Cohesion vs. Coupling

- Business functionality should drive system design to promote ease of change, prioritizing cohesion over mere technological alignment.

Alternative Architecture Example

- A Customer service-centric approach encapsulates UI, application logic, and data within a single service, enhancing local change efficiency.

Decentralized Data Ownership

- Microservices should not share databases. This enhances independent deployability by ensuring stable service interfaces and reducing coupling.

More Free Book



Scan to Download



Listen It

Advantages of Microservices

- Microservices enhance flexibility, independent parallel development, and technology mixing. However, they come with challenges and require strategic planning for decomposition from monolithic architectures.

Challenges with Service-Oriented Architecture

- Communication delays, network failures, and complex interactions can hinder operations compared to monolithic systems. Managing these issues is key to realizing the benefits of microservices.

User Interfaces and Technology Considerations

- Maintaining a monolithic UI is a mistake when adopting microservices. Teams are encouraged to think about the user interface in a similar way to services.
- Caution is advised against introducing new technologies blindly during the adoption of microservices; understanding existing system complexities is crucial.

More Free Book



Scan to Download



Listen It

Size and Historical Context of Microservices

- Service size is less important than functionality and cohesion. Size can be subjective; focus should be on managing complexity and defining boundaries.
- The term "microservices" emerged from previous discussions of smaller, easily replaceable services in the context of service-oriented architectures.

Business Domain Alignment

- Microservices modeled around business domains facilitate alignment between IT and business operations, promoting ownership and transparency.

Monoliths and Their Challenges

- Monoliths, whether single-process or distributed, face concerns regarding coupling and deployability, which can lead to delivery contention.
- Monoliths have advantages such as simpler deployment and testing but often hinder agility and team ownership.

Coupling and Cohesion Dynamics

More Free Book



Scan to Download



Listen It

- High cohesion and low coupling are desirable for stability and rapid modification. Cohesion relates to related code changes appearing together, while coupling pertains to how changes in one area affect another.

Information Hiding Principle

- Information hiding limits exposure of implementation details, fostering independent changes without affecting broader system components.

Deployment Strategies

- Strategies to reduce deployment coupling through independent microservices can lead to smaller, safer releases, enhancing the potential for continuous delivery.

Domain Model Considerations

- Domain-driven design helps structure services around business needs, emphasizing the use of aggregates and bounded contexts as service boundaries.

More Free Book



Scan to Download



Listen It

Conclusion and Planning for Migration

- Microservices should be pursued with clear objectives rather than as an automatic response to trends. Organizations should assess their specific needs and consider alternatives before transitioning to microservices.

Key Questions for Migration

1. What are the desired outcomes aligned with business goals?
2. Have alternatives to microservices been considered?
3. What strategies will be employed to facilitate this significant change?

More Free Book



Scan to Download



Listen It

Example

Key Point: Business functionality should guide system design, fostering high cohesion and low coupling for rapid adaptability.

Example: Imagine you're leading a development team for an e-commerce platform, and a major client requests a new feature that allows them to offer seasonal discounts on select products. If your architecture is designed around cohesive business domains, you could quickly implement this change in one coherent service that contains all necessary components—UI for the discount selection, backend processing to apply these changes, and direct access to the database for real-time updates. This setup enables your team to work in parallel on different functionalities without being hindered by complex interactions across distinct layers. Alternatively, if your system is merely cobbled together based on technology alone, integrating this new feature could become a slow and painful process, invoking the need for multiple teams to coordinate extensive changes across various layers, stalling your capability to respond swiftly to market needs.



Critical Thinking

Key Point:Business Domain Alignment

Critical Interpretation:Newman's emphasis on aligning microservices with business domains as crucial for effective IT operations could be contested. While he argues it enhances ownership and transparency, one might question whether this alignment genuinely leads to improved software delivery or simply introduces complexity in managing domain boundaries. Critics like Martin Fowler suggest that while business alignment is valuable, the reality of system evolution often necessitates compromise in domain boundaries to accommodate technical realities, indicating that the author's perspective may overlook valuable complexity management strategies present in more hybrid architectural approaches.

More Free Book



Scan to Download



Listen It

Chapter 2 Summary :



Monitoring Transition Success

To gauge the effectiveness of the transition to microservices, it's essential to have clear indicators for success. Revisiting these indicators often prompts companies to reassess their commitment to microservice architectures.

Reasons to Choose Microservices

While goals may vary by organization, common motivations for adopting microservices include:

Improve Team Autonomy

More Free Book



Scan to Download



Listen It

Creating autonomous teams has proven beneficial across various industries. Empowering smaller teams allows them to collaborate closely without excessive bureaucracy, fostering scalability and efficiency.

Reduce Time to Market

Microservices facilitate faster updates and deployments, allowing companies to deliver functionality to customers more swiftly. Agencies should evaluate all release processes to identify inefficiencies that may delay delivery.

Scale Cost-Effectively for Load

Individual microservices can be scaled independently, permitting organizations to manage operational costs more effectively based on current demand.

Improve Robustness

Decoupling features into microservices helps prevent system-wide failures from impacting all functionalities, reinforcing resilience.

More Free Book



Scan to Download



Listen It

Scale the Number of Developers

With clearly defined service boundaries, organizations can effectively allocate more developers to projects, alleviating bottlenecks that typically hinder collaboration.

Embrace New Technology

Microservices permit varied technology stacks, allowing teams to implement the best technology for specific services without overhauling the entire architecture.

Caution Against Reuse as a Goal

Focusing on reuse may lead to increased complexity and coordination, potentially delaying delivery. Tracking objectives like quicker time to market may yield better outcomes than solely prioritizing reuse.

Situations Where Microservices May Not Be Ideal

Unclear Domain

More Free Book



Scan to Download



Listen It

Choosing inappropriate service boundaries may lead to instability and increased costs. It's prudent to cement a clear understanding of the domain before pursuing decomposition.

Startups

Startups may benefit from microservices during the scaling stage but should focus initially on achieving product-market fit rather than complex architectures.

Customer-Installed Software

Deploying microservices to self-managed customer environments introduces significant operational complexity that customers may not be equipped to handle.

Lack of Clear Purpose

Adopting microservices without defined goals can lead to failures. A sound strategy should underlie any transition.

Trade-offs and Prioritization

Navigating simultaneous changes can complicate

More Free Book



Scan to Download



Listen It

decision-making; thus, prioritizing must occur, focusing initially on scaling or autonomy as major goals without overwhelming teams with extraneous projects.

Organizational Change Management

Building consensus around the transition is critical. Drawing from established models of organizational change, such as Kotter's eight-step process, can facilitate smoother transitions to microservices.

Establishing a Sense of Urgency

Communicate the importance of the shift and the need for successful microservices to avoid being another good idea left unpursued.

By understanding initiatives that need to be implemented and their impact, organizations can execute a successful transition to microservices, ensuring all stakeholders are aligned and prepared for the journey ahead.

More Free Book



Scan to Download



Listen It

Example

Key Point: Establishing clear indicators of transition success is vital for monitoring microservices effectiveness.

Example: Imagine you are leading a tech team that recently transitioned to microservices. To navigate this change, you start by defining key performance indicators (KPIs) such as deployment frequency and system uptime. As the weeks go by, you regularly review these metrics in team meetings, celebrating improvements and identifying areas needing attention. For example, seeing a significant reduction in deployment times emphasizes your team's ability to respond to customer feedback swiftly. This iterative analysis not only reassures your stakeholders of the transition's success but also reinforces your team's focus on continuous improvement.

More Free Book



Scan to Download



Listen It

Critical Thinking

Key Point: The Importance of Clear Success Indicators in Microservices Implementation

Critical Interpretation: While Sam Newman emphasizes the necessity of clear indicators for success in transitioning to microservices, it is vital for readers to critically evaluate this assertion. Indicators can vary across organizations, and what works for one may not be suitable for another, leading to potential misalignment of expectations. Some sources, such as 'Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation' by Jez Humble and David Farley, show that flexibility in measuring success can provide a more nuanced understanding of transition outcomes. Thus, organizations should approach Newman's viewpoint with caution, ensuring that their chosen indicators align with their specific circumstances and business goals.

More Free Book



Scan to Download



Listen It

Chapter 3 Summary :

Responsibilities in Transitioning to Microservices

The organization can reassign responsibilities to handle various areas such as frontend development, backend development, testing, and incident management. A big-picture view helps to visualize these shifts, but collaboration with the team is essential to ascertain feasibility and to implement changes incrementally. Starting with simplifying the provisioning of test environments can be an initial focus.

Changing Skills

Evaluating and enhancing team skills is critical in the transition to microservices. Self-assessment can help identify gaps and areas of improvement, allowing individuals to set personal goals for skill enhancement. Keeping these self-assessment results private encourages personal growth without the pressure of competition. Team skill maps can highlight systemic gaps, enabling targeted group learning initiatives.

More Free Book



Scan to Download



Listen It

Assessing Transition Success

To determine if the transition to microservices is successful, both quantitative and qualitative metrics should be established. Regular checkpoints should be incorporated into the process to reassess progress and direction. Key questions during these checkpoints should include evaluating targets, gathering feedback, and deciding if a change of strategy is necessary.

Avoiding Sunk Cost Fallacy

It's important to remain adaptable throughout the transition, avoiding the sunk cost fallacy by reassessing strategies regularly based on evidence and feedback. Maintaining an iterative approach allows for manageable and reversible changes as the team learns and evolves.

Install Bookey App to Unlock Full Text and Audio

More Free Book



Scan to Download



Listen It



Scan to Download



Why Bookey is must have App for Book Lovers



30min Content

The deeper and clearer interpretation we provide, the better grasp of each title you have.



Text and Audio format

Absorb knowledge even in fragmented time.



Quiz

Check whether you have mastered what you just learned.



And more

Multiple Voices & fonts, Mind Map, Quotes, IdeaClips...

Free Trial with Bookey



Chapter 4 Summary :

Chapter 4 Summary: Migration Strategies for Microservices

Transitioning to Microservices

- The migration to microservices can be facilitated by breaking the monolith incrementally, often using user interface (UI) decomposition as a guide.
- Mobile applications can be adapted similarly to web interfaces, but face challenges with app store approvals and monolithic structures requiring full redeployments for any changes.
- Dynamic updates to native mobile applications can be achieved through server-side changes, enhancing flexibility.

Micro Frontends

- The shift to microservices is complemented by the adoption of micro frontends, which break UI components into independent parts that can be developed in isolation. This

More Free Book



Scan to Download



Listen It

necessitates strategies for integration and dependency management across different frameworks like Vue and React.

Branch by Abstraction Pattern

- The "Branch by Abstraction" pattern allows for gradual code changes without disrupting ongoing development. This involves creating abstractions for existing functionalities allowing new implementations to coexist until fully validated before deployment.

Parallel Run Pattern

- The "Parallel Run" pattern allows old and new systems to operate simultaneously, providing a method for verifying that the new implementation performs correctly without cutting over fully until confident in its reliability.
- This approach includes both functional and non-functional testing, ensuring that performance metrics like response times are also validated.

Decorating Collaborator and Change Data Capture Patterns

More Free Book



Scan to Download



Listen It

- "Decorating Collaborator" facilitates adding functionality based on events in the monolith without changing its code, while "Change Data Capture" reacts to alterations in the underlying data store, allowing integration with microservices through reacting to data changes.

Database Management Strategies

- Addressing database coupling is crucial when transitioning to microservices. The chapter discusses various strategies:

-

Shared Database

: Generally discouraged unless for read-only reference data.

-

Database Wrapping Service

: Creates a thin service layer over the database to cloak complexity, limiting direct database interactions.

-

Database-as-a-Service Interface

: Exposes a dedicated read-only database for external use, enabling controlled access without exposing internal structures.

Implementing Views and Change Data Capture

More Free Book



Scan to Download



Listen It

- The use of database views can mitigate exposure to underlying schema complexities, though they have limitations.
- A dedicated mapping engine for change data capture can maintain synchronization between internal and external databases, ensuring data visibility while keeping internal architecture abstracted.

Conclusion

- Successful migration to a microservices architecture requires a thoughtful, multi-faceted approach that balances incremental change with the need for stability, data integrity, and continued operational efficiency. Implementing structured patterns can facilitate this transit.

More Free Book



Scan to Download



Listen It

Chapter 5 Summary :

Chapter 5 Summary: Managing Data in Microservices

Introduction

The shift from a monolithic architecture to microservices involves critical decisions about data ownership and management. Some data will travel with the extracted services, while other data remains in the monolith.

Data Management Approaches

1.

Aggregate Exposing Monolith

- A new service may require data managed in the monolith to function. Accessing this data should be through a well-defined service interface to expose necessary information without tightly coupling systems. This approach lays the foundation for discovering future service boundaries.

More Free Book



Scan to Download



Listen It

2.

Change Data Ownership

- When data should be controlled by a new microservice (like invoice data), it must be extracted from the monolith. This requires altering the monolith's operations to treat the new service as the source of truth, necessitating careful management of foreign-key constraints and transactional integrity.

Synchronizing Data

- Maintaining data consistency during migrations involves critical decisions:

- Data could be kept in sync through shared databases temporarily, but this approach could lead to long-term pain if not transitioned properly.

3.

Synchronize Data in Application

- For scenarios requiring data to be kept in sync, using a transitional approach—like a change data capture technique—could help. This is essential when switching between systems while maintaining data integrity.

More Free Book



Scan to Download



Listen It

4.

Tracer Write

- This incremental migration pattern allows for having dual sources of truth during the transition. It permits gradual synchronization of data, ensuring that all changes are accounted for without a disruptive big bang migration.

Implementation Patterns

1.

Splitting the Database First or Code

- The sequence of splitting code and database can be flexible:

-

Database First

: Can reveal performance issues but may delay the benefits from a microservice.

-

Code First

: Provides quicker improvements but risks incomplete separation if not followed with database decomposition.

2.

More Free Book



Scan to Download



Listen It

Monolith as Data Access Layer

- When first introducing microservices, creating APIs on the monolith for data access allows separation without losing data management capabilities in the short term.

3.

Multischema Storage

- New functionalities should manage their own storage separately from the monolith to prevent growth in the monolithic database.

4.

Split Database and Code Together

- Although tempting, this approach poses higher risks and is discouraged in favor of a more incremental strategy.

Schemas and Data Access

- Consider using patterns like repository per bounded context and database per bounded context to create separation and modular design within databases.

Final Thoughts

More Free Book



Scan to Download



Listen It

- Choosing the right approach to data management and synchronization during a transition to microservices requires careful planning and consideration of the specific context, potential technical challenges, and organizational goals.

More Free Book



Scan to Download



[Listen It](#)

Chapter 6 Summary :

Section	Summary
Introduction	This chapter discusses the challenges of data management during the transition from monolithic systems to microservices, focusing on schema evolution.
Incremental Migration and Data Separation	Transitioning involves splitting the monolithic data into separate tables for each service, starting with the existing schema before creating distinct schemas.
Foreign-Key Relationships	Foreign-key relationships help with data integrity but become less relevant across different databases; managing ownership and changes in shared data is critical.
Pattern: Move Foreign-Key Relationship to Code	The shift from a centralized table schema to microservice-oriented design requires adjustments in data retrieval, avoiding direct database joins.
Handling Data Consistency	Strategies to maintain data consistency across services in the absence of a unified schema include graceful data handling and soft deletion.
Shared Static Data Management	Discussion on whether to duplicate static reference data across services or centralize it, with options for dedicated schemas, static libraries, or services.
Transactions in Microservices	The chapter addresses the challenges of maintaining atomicity in operations post-split, highlighting the complexity of distributed transactions.
Sagas as an Alternative	Sagas are introduced for handling long-lived transactions across services, characterized by orchestrated and choreographed approaches for process management.
Implementation Considerations	Effective saga management requires defining compensating transactions and reorganizing transaction steps to reduce rollback scenarios.
Conclusion	The chapter offers guidance on managing transactions and data consistency, emphasizing adaptability, incremental migration, and clarity in data ownership.

Chapter 6 Summary: Refactoring Data in Microservices

Introduction

In this chapter, the process of splitting monolithic systems into microservices is discussed, focusing on data

More Free Book



Scan to Download



Listen It

management challenges, particularly during schema evolution.

Incremental Migration and Data Separation

- Transitioning from a monolithic Catalog and Warehouse to microservices involves splitting data stored in a single table into separate tables for each service.
- This separation should initially occur within the existing schema before distinct schemas are established.

Foreign-Key Relationships

- Introducing foreign-key relationships in the existing schema helps maintain data integrity but is less useful when data is split across databases.
- When multiple services manipulate a single column (e.g., Customer Status), determining ownership and managing

Install Bookey App to Unlock Full Text and Audio

More Free Book



Scan to Download



Listen It

Ad



Scan to Download



App Store
Editors' Choice



22k 5 star review

Positive feedback

Sara Scholz

...tes after each book summary
...erstanding but also make the
...and engaging. Bookey has
...ding for me.

Fantastic!!!



I'm amazed by the variety of books and languages
Bookey supports. It's not just an app, it's a gateway
to global knowledge. Plus, earning points for charity
is a big plus!

Masood El Toure

Fi



Ab
bo
to
my

José Botín

...ding habit
...o's design
...ual growth

Love it!



Bookey offers me time to go through the
important parts of a book. It also gives me enough
idea whether or not I should purchase the whole
book version or not! It is easy to use!

Wonnie Tappkx

Time saver!



Bookey is my go-to app for
summaries are concise, ins
curated. It's like having acc
right at my fingertips!

Awesome app!



I love audiobooks but don't always have time to listen
to the entire book! bookey allows me to get a summary
of the highlights of the book I'm interested in!!! What a
great concept !!!highly recommended!

Rahul Malviya

Beautiful App



This app is a lifesaver for book lovers with
busy schedules. The summaries are spot
on, and the mind maps help reinforce wh
I've learned. Highly recommend!

Alex Walk

Free Trial with Bookey



Chapter 7 Summary :

Chapter 7 Summary: Challenges in Microservice Architectures

Downsides of Orchestration

Orchestration can couple services tightly, leading to high domain coupling where controllers, like the Order Processor, need knowledge of all associated services. This may cause services to become anemic as logic centralizes in the orchestrator rather than remaining with individual services. To mitigate centralization, distributing orchestrators for different flows is recommended.

BPM Tools

Business Process Modeling (BPM) tools aim to allow non-developers to define processes, but often developers use them, leading to complications such as difficulty in version control and testing issues. Developers should utilize familiar coding tools instead. Nonetheless, there are efforts to create

More Free Book



Scan to Download



Listen It

more developer-friendly BPM tools, such as Camunda and Zeebe.

Choreographed Sagas

Choreographed sagas distribute the saga responsibility among services using event-driven collaboration, reducing coupling as services only react to events rather than communicate directly. While this fosters independence, it complicates tracking the saga's state and necessitates correlation IDs for state management.

Mixing Orchestration and Choreography

It's feasible to mix both methods within a single saga or across different processes, balancing the benefits of orchestration and the loose coupling of choreography. Clear documentation is essential to avoid confusion and understand failure modes.

Choosing Between Choreography and Orchestration

Orchestration is manageable when a single team is responsible for the entire saga; however, choreography is

More Free Book



Scan to Download



Listen It

preferred in multi-team projects due to its decentralized nature. While distributed transactions are challenging, sagas provide a more efficient means of handling long-running processes with explicit models.

Growing Pains in Microservices

Microservices introduce complexity, especially with increasing numbers of services. Common challenges include the need for robust ownership structures, the risks of breaking changes, reporting difficulties, monitoring/troubleshooting complexities, and variations in developer experience.

Ownership at Scale

As service count and team sizes grow, clearly defined ownership becomes crucial. Strong code ownership is recommended for large-scale systems to ensure accountability and maintain coherence amidst rapid expansion.

Handling Breaking Changes

More Free Book



Scan to Download



Listen It

To prevent breaking changes, maintain explicit contracts and schemas, use consumer-driven contracts to define expectations, and allow consumers time to adjust to changes. Implementing rollback mechanisms can mitigate the impact of unintentional breaks.

Reporting and Monitoring

With microservices, the traditional monolithic reporting approach becomes inadequate due to data segregation. Implementing dedicated reporting databases and effective monitoring solutions, including log aggregation and tracing tools, is key to maintain systems efficiently.

Local Developer Experience

Developer efficiency decreases as the number of services increases. Solutions include stubbing out services and incorporating remote development tools, such as Telepresence, to enhance local development workflows.

Orphaned Services

Orphaned services in microservices can lead to operational

More Free Book



Scan to Download



Listen It

risks. An inventory of services and a defined ownership structure can help mitigate this issue, ensuring services are maintained and knowledge does not fade.

Summary

Managing microservice architectures comes with inherent challenges, from operational complexities to strategic decision-making. The chapter highlights that while navigating these issues can be complex, understanding potential pitfalls enables teams to prepare better and adapt their practices. Adopting a balanced approach to ownership, monitoring, and decision-making can enhance the resilience and effectiveness of microservices.

More Free Book



Scan to Download



Listen It



Read, Share, Empower

Finish Your Reading Challenge, Donate Books to African Children.

The Concept



This book donation activity is rolling out together with Books For Africa. We release this project because we share the same belief as BFA: For many children in Africa, the gift of books truly is a gift of hope.

The Rule



Earn 100 points



Redeem a book



Donate to Africa

Your learning not only brings knowledge but also allows you to earn points for charitable causes! For every 100 points you earn, a book will be donated to Africa.

Free Trial with Bookey



Best Quotes from Monolith to Microservices by Sam Newman with Page Numbers

[View on Bookey Website and Generate Beautiful Quote Images](#)

Chapter 1 | Quotes From Pages 16-38

1. Any organization that designs a system... will inevitably produce a design whose structure is a copy of the organization's communication structure.
2. If we want to make it easier to make changes, instead we need to change how we group code—we choose cohesion of business functionality, rather than technology.
3. Microservices buy you options.
4. We want to think of our services as end-to-end slices of business functionality, that where appropriate encapsulate the UI, application logic, and data storage.
5. You need a clear understanding as to what you are hoping to achieve.
6. Don't share databases, unless you really have to. And even

More Free Book



Scan to Download



[Listen It](#)

then do everything you can to avoid it.

7. We're seeing true technology organizations totally combine these previous disparate organizational silos.

8. The act of hiding a database behind a well-defined service interface allows the service to limit the scope of what is exposed.

Chapter 2 | Quotes From Pages 39-60

1. It is all about your people, and catching them

doing things right, and providing them with the

confidence, the motivation, the freedom and desire

to achieve their true potential." —John Timpson

2. Autonomy works at the smaller scale too, and most modern

organizations I work with are looking to create more

autonomous teams within their organizations... If done

right, team autonomy can empower people, help them step

up and grow, and get the job done faster.

3. Reusability is not a direct outcome people want. Reuse is

something people hope will lead to other benefits.

4. If you're working in an environment where the developers

More Free Book



Scan to Download



Listen It

are used to working 9-5, not being on call, have never worked in a support or operations environment, and wouldn't know their SSH from their elbow, then this is a great way to alienate your staff and lose a lot of people.

5. Incremental approach will help you learn about microservices as you go, and will also limit the impact of getting something wrong (and you will get things wrong!).
6. It is really important to note that the extraction of a microservice can't be considered complete until it is in production and being actively used.
7. The important thing here is to recognize that some things are more important than others. Otherwise, you can't properly prioritize.
8. Don't do microservices just because everyone else is doing it; it's crucial to have a clear idea of what you're trying to achieve.
9. Break the big journey into lots of little steps. Each step can be carried out and learned from.
10. If you make a small change, it's much easier to spot (and

More Free Book



Scan to Download



Listen It

fix) a problem you create.

11. The snapshot of Spotify's organizational structure reflected changes it had carried out to solve its problems. Copy that flexible, questioning attitude in how you do things, and try new things but make sure the changes you apply are rooted in an understanding of your company, its needs, its people, and its culture.

Chapter 3 | Quotes From Pages 61-85

1. You should try defining some measures that can be tracked and can help you answer these questions.
2. Whatever our data shows us, it's people who build the software, and it's important that their feedback is included in measuring success.
3. The goal should be incremental creation of new microservices, and getting them deployed as part of your production solution so that you start learning from the experience.
4. If you make each step a small one, it becomes easier to avoid the pitfalls of the sunk cost fallacy.

More Free Book



Scan to Download



Listen It

5. Be inspired by case studies from other companies, but don't be surprised if slavishly copying someone else's answers to their problems ends up not working out well for you.

More Free Book



Scan to Download



Listen It



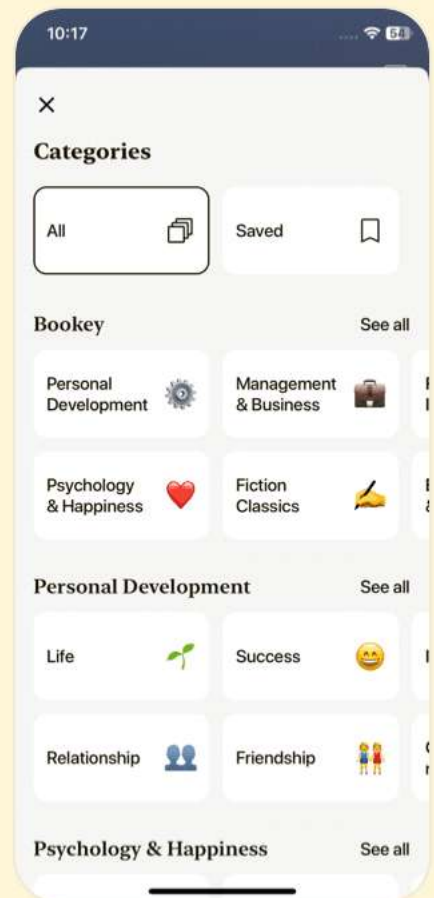
Download Bookey App to enjoy

1 Million+ Quotes

1000+ Book Summaries

Free Trial Available!

Scan to Download



Chapter 4 | Quotes From Pages 86-113

- 1.Many organizations have dealt with this by allowing them to make dynamic changes to an existing native mobile application without having to resort to deploying a new version of a native application.
- 2.Micro Frontends really are just about breaking down a user interface into different components that can be worked on independently.
- 3.Branch by abstraction consists of five steps: 1. Create an abstraction for the functionality to be replaced...
- 4.The ability for us to switch back to our previous implementation if our new service isn't behaving is useful, but is there a way we could do that automatically?
- 5.Using a Spy allows us to verify that this side effect (the sending of the email) would be triggered when the sendNotification call is received by the service.
- 6.Splitting a database apart is far from a simple endeavor, however.



- 7.The more of them you have, the harder it can be to understand how your system actually works.
- 8.Eventually, after a month, we switched over to using our system as the source of truth for the calculations.
- 9.This approach allows us to control what is shared and what is hidden.
- 10....the use of a wrapping service allows us to control what is shared and what is hidden.

Chapter 5 | Quotes From Pages 114-137

- 1.We've just put a variety of different bandages on a big, shared database.
- 2.When exposing an aggregate from the monolith, we want to think in the same terms.
- 3.By exposing an API to employee-related data, we've already gone a long way to understanding what the needs of the consumers of the new Employee service might be.
- 4.If your newly extracted service encapsulates the business logic that changes some data, that data should be under the new service's control.

More Free Book



Scan to Download



Listen It

5. Wanting a single source of truth is a totally rational desire.
6. Having the database mapping code colocated inside the code for a given context can help us understand what parts of the database are used by what bits of code.
7. If implemented correctly, both data sources should always be in sync, offering us significant benefits in situations where we need fast switching.
8. The tracer write allows for a phased switchover, reducing the impact of each release in exchange for being more tolerant of having more than one source of truth.
9. We'll be exploring this topic in great length later in this chapter.
10. The challenges of using databases as a method of integrating multiple services.

Chapter 6 | Quotes From Pages 138-160

1. Remember, we want, where possible, to keep the state machines for our domain entities inside a single service boundary.
2. We could also consider using events to let consumers know

More Free Book



Scan to Download



Listen It

when this data has changed, as shown in Figure 4-45.

3. As developers, we often react badly when we see duplication. We worry about the extra cost of managing duplicate copies of information... But sometimes duplication is the lesser of two evils.
4. If you have pieces of state that you want to manage in a truly atomic and consistent way, and you cannot work out how to sensibly get these characteristics without an ACID-style transaction, then leave that state in a single database, and leave the functionality that manages that state in a single service (or in your monolith).
5. Sagas... can coordinate multiple changes in state, but avoids the need for locking resources for long periods of time.
6. With a single schema, I wouldn't be able to delete a row in the Albums table if there was a reference to that row in the Ledger table. My schema was enforcing data consistency.

More Free Book



Scan to Download



Listen It



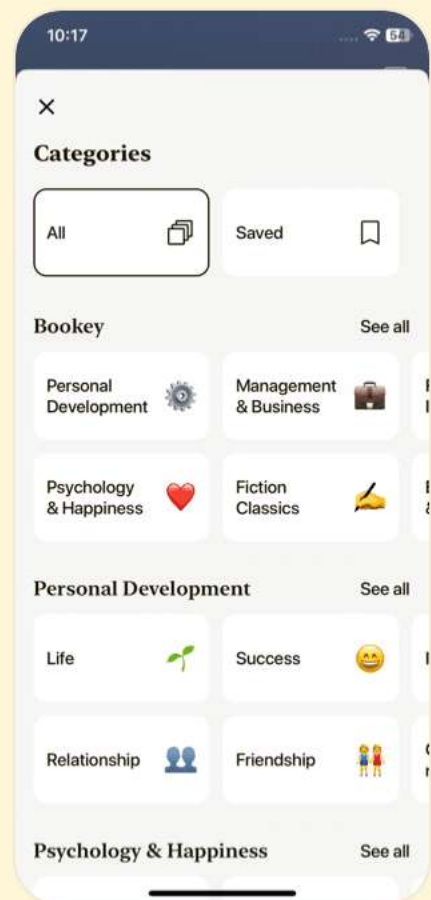
Download Bookey App to enjoy

1 Million+ Quotes

1000+ Book Summaries

Free Trial Available!

Scan to Download



Chapter 7 | Quotes From Pages 161-186

- 1....if logic has a place where it can be centralized, it will become centralized!
- 2....your developers are going to be implementing your business processes, let them use tooling that they know and understand and is fit for their workflows.
- 3.If orchestration is command-and-control, choreographed sagas represent a trust-but-verify architecture.
- 4....it often becomes harder to work out what is going on.
- 5....ensure you have different services playing the role of the orchestrator for different flows.
- 6.Without a central place to interrogate around the status of a saga is a big problem.
- 7.Don't think of adopting microservices as flipping a switch; think about it as turning a dial.
- 8...the general advice I give regarding orchestration versus choreography... when one team owns implementation of the entire saga.
- 9...the cost of detangling a distributed monolith is much



higher.

10. You need to gather information about what your system is doing, allowing us to after the fact ask questions of the data that we didn't know we'd have to ask.

More Free Book



Scan to Download



Listen It



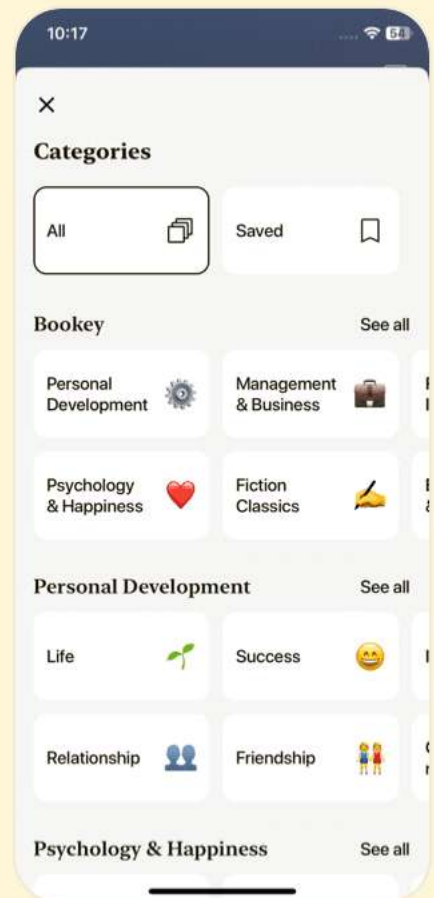
Download Bookey App to enjoy

1 Million+ Quotes

1000+ Book Summaries

Free Trial Available!

Scan to Download



Monolith to Microservices Questions

[View on Bookey Website](#)

Chapter 1 | Q&A

1.Question

What does the change in functionality to allow customers to specify their favorite genre of music illustrate about team structure and communication?

Answer:It highlights the challenges of a three-tiered architecture where changes must be managed across different teams (UI, Backend, Database) in a synchronized manner. This requires effective communication and coordination, thus embodying Conway's Law, which suggests that the organization of teams impacts system architecture.

2.Question

Why is the three-tier architecture prevalent according to Conway's Law?

Answer:It is prevalent because organizations typically structure themselves around core competencies, resulting in

More Free Book



Scan to Download



[Listen It](#)

systems that reflect those divisions. This leads to low cohesion in business functionality despite high cohesion in related technologies.

3.Question

What is the underlying issue with a three-tier architecture when it comes to making functionality changes?

Answer:The issue is that functionality is spread across multiple tiers, increasing the likelihood that a change will require coordination across these layers, which complicates deployment and can lead to errors.

4.Question

How does the proposed alternative architecture using a Customer service simplify changes?

Answer:It simplifies changes by encapsulating functionality related to customer data within a single service. This means that changes can be made in a localized manner without affecting other tiers, allowing for quicker and easier updates.

5.Question

What does it mean to 'own their own data' in the context of microservices?

More Free Book



Scan to Download



Listen It

Answer:It means that each microservice should manage and control its own data, avoiding shared databases. This ensures that each service can evolve independently without being affected by changes in other services, thereby enhancing stability and flexibility.

6.Question

What are some advantages of adopting a microservices architecture?

Answer:The key advantages include greater flexibility, improved scalability, the ability to deploy updates independently without affecting the entire system, and the opportunity to utilize diverse technology stacks across services.

7.Question

What are some challenges associated with microservices despite their advantages?

Answer:Challenges include increased complexity of managing distributed systems, potential communication delays, requiring robust error handling due to network

More Free Book



Scan to Download



Listen It

failures, and the necessity of achieving a consistent view of data across services.

8.Question

How should the size of a microservice be determined?

Answer:The size should not be rigidly defined; rather, it should focus on how well a service can encapsulate a cohesive functional domain. It's more important to pursue clear boundaries and maintain manageable service counts for effective collaboration.

9.Question

What should be considered before migrating to a microservice architecture?

Answer:Organizations should have a clear understanding of their goals for migration, evaluate alternative solutions, and ensure that the transition aligns with business objectives to avoid unnecessary complexity and confusion.

10.Question

What role does domain-driven design play in the architecture of microservices?

Answer:Domain-driven design helps create services that

More Free Book



Scan to Download



Listen It

align closely with real-world business contexts and workflows, ensuring that the structure of services promotes high cohesion and minimal coupling.

Chapter 2 | Q&A

1.Question

How will you know if the transition to microservices is working?

Answer: You'll need to evaluate the goals and objectives you set out at the start. Key indicators could include improved team autonomy, reduced time to market, and effective scaling of systems. Regularly checking against these objectives will help you assess progress.

2.Question

What is one major reason companies choose microservices?

Answer: Improving team autonomy is a common motivation. Teams can work more independently on microservices they own, thus reducing dependencies and bureaucracy, enabling

More Free Book



Scan to Download



Listen It

faster decision making.

3.Question

Can you explain the concept of 'Autonomy' in a microservices architecture?

Answer:Autonomy allows teams to take ownership of specific services, which empowers them to make decisions and implement changes without excessive reliance on other teams. This leads to faster development cycles and innovation.

4.Question

What is a risk of adopting microservices too early?

Answer:A major risk is overestimating understanding of your domain. Incorrect service boundaries can lead to high costs due to complexity. Teams may need to revisit and combine services back into a monolith if boundaries are poorly defined.

5.Question

How does microservices affect time to market?

Answer:Microservices can reduce time to market because changes to individual services can be made, tested, and

More Free Book



Scan to Download



Listen It

deployed independently, without needing to synchronize with a larger release cycle. This allows businesses to deliver functionality to customers more quickly.

6.Question

Why might startups want to avoid microservices initially?

Answer:Startups are often in a stage of discovery, figuring out product-market fit. Microservices are better suited for established systems where the boundaries are clear.

Attempting to start with microservices may complicate their agile nature.

7.Question

What is an effective approach to scaling operational costs in a microservice architecture?

Answer:Individual microservices can be scaled independently, allowing the organization to invest resources only where there is increased demand while scaling down less utilized services, ultimately optimizing operational costs.

8.Question

How can organizations ensure robustness in a microservices architecture?

More Free Book



Scan to Download



Listen It

Answer:Robustness can be achieved by designing independent services to respond to failures without affecting the entire system. Implementing redundancy and monitoring capabilities can also enhance overall system robustness.

9.Question

What does the author suggest about making changes during a transition to microservices?

Answer:He advises to make changes incrementally. Instead of a big bang approach, gradually chipping away allows for learning and adjustment, reducing the risk and impact of mistakes.

10.Question

How can companies ensure smooth change management when adopting microservices?

Answer:Using Kotter's eight-step process for implementing change can be beneficial. Establish a sense of urgency, build a guiding coalition, communicate the change, and empower employees are key steps in managing change effectively.

11.Question

What is the significance of establishing a clear vision

More Free Book



Scan to Download



Listen It

when transitioning to microservices?

Answer:A clear vision aligns teams and stakeholders on the purpose and desired outcomes of transitioning to microservices, ensuring that everyone understands what is being aimed for and maintains focus throughout the process.

Chapter 3 | Q&A

1.Question

What is the importance of dividing responsibilities among team members during the transition to microservices?

Answer:Dividing responsibilities allows for a clearer focus on specific tasks and enables an incremental approach to the transition. This means each team can concentrate on specific areas of expertise, making the overall process more feasible and manageable.

2.Question

How can self-assessment help team members in developing their skills?

Answer:Self-assessment encourages team members to

More Free Book



Scan to Download



Listen It

evaluate their skills privately, allowing them to identify areas for improvement without the fear of judgment from their peers. This can lead to personalized development plans that cater to individual growth while contributing to the team's overall skill enhancement.

3.Question

Why is it crucial to include both quantitative and qualitative measures when assessing the success of a transition to microservices?

Answer:Quantitative measures provide hard data on progress, while qualitative measures capture the feelings and experiences of team members. This combination allows for a more comprehensive understanding of the transition's impact and helps to identify areas that may need adjustment.

4.Question

What is the sunk cost fallacy, and how can it affect decision-making during a transition to microservices?

Answer:The sunk cost fallacy occurs when people continue an endeavor due to previously invested resources, despite evidence indicating that the course of action may be flawed.

More Free Book



Scan to Download



Listen It

This can lead to sticking with ineffective strategies out of emotional attachment rather than making rational decisions based on current evidence.

5.Question

How can the concept of incremental changes mitigate the risk of a failed transition during the shift to microservices?

Answer:By making small, manageable changes instead of sweeping reforms, teams can minimize disruptions and quickly address any unanticipated issues that arise. This approach also allows for adjustments based on feedback and learning, making the transition smoother and more adaptable.

6.Question

What does the term 'strangler fig pattern' refer to in the context of migrating to microservices?

Answer:The strangler fig pattern refers to incrementally replacing an existing monolithic application with microservices. It allows new services to grow and take over specific functionalities while keeping the old system operational until the transition is complete.

More Free Book



Scan to Download



Listen It

7.Question

Why is it essential to conduct regular checkpoints during the transition to microservices?

Answer:Regular checkpoints provide opportunities for teams to reflect on progress, re-evaluate goals, and make necessary adjustments based on what is or isn't working. They help ensure the transition is on track and allows for course corrections if needed.

8.Question

What role does qualitative feedback play in understanding team dynamics during a transition to microservices?

Answer:Qualitative feedback offers insight into the team's morale, engagement, and challenges, which cannot be captured by metrics alone. Understanding how team members feel during the transition can uncover issues that need to be addressed to support their well-being and performance.

9.Question

How can embracing a culture of constant improvement

More Free Book



Scan to Download



Listen It

benefit a team transitioning to microservices?

Answer: A culture of constant improvement encourages teams to be open-minded, adaptative, and willing to experiment with new ideas and processes. This mindset fosters innovation and resilience, ultimately making the transition smoother and more effective.

10.Question

What should teams prioritize when migrating existing code to microservices?

Answer: Teams should prioritize understanding the existing monolithic codebase, identifying well-factored components, and ensuring that any migration does not disrupt ongoing operations. Incremental migration allows teams to learn and adjust while minimizing risks.

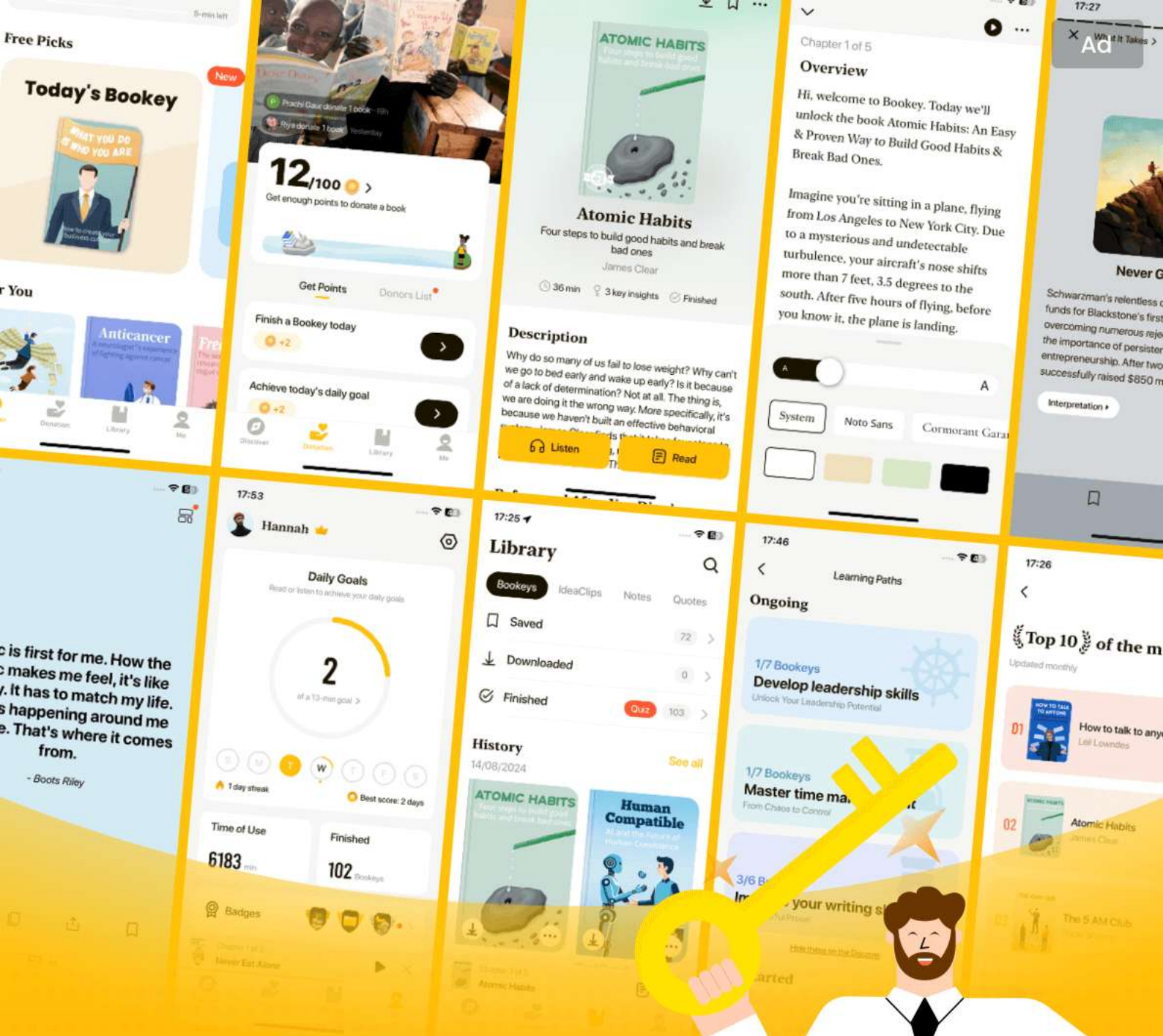
More Free Book



Scan to Download



Listen It



World' best ideas unlock your potential

Free Trial with Bookey



Scan to download



Chapter 4 | Q&A

1.Question

What is one advantage of decomposing a user interface into clean widgets when migrating to microservices?

Answer:Decomposing a UI into clean widgets facilitates incremental migration, making it easier to manage changes without impacting other teams. For example, separate modules with clear ownership allow for focused updates while maintaining overall system integrity.

2.Question

How do mobile applications present challenges during migration to microservices?

Answer:Mobile apps often require submitting new versions to app stores for deployment, which introduces delays.

Unlike web applications, where changes are instantly available, native apps necessitate user downloads of updated versions.

3.Question

What is the branch by abstraction pattern, and how does

More Free Book



Scan to Download



Listen It

it help with gradual migration?

Answer: The branch by abstraction pattern enables incremental changes to the code base by introducing an abstraction layer that allows old and new implementations to coexist. This minimizes disruption by allowing developers to work in tandem without long-lived branches.

4.Question

Define the parallel run pattern and its purpose in a microservice migration scenario.

Answer: The parallel run pattern involves running both the old and new implementations of functionality simultaneously to verify equivalence, ensuring that the new microservice behaves as expected before fully transitioning users.

5.Question

In what situation might change data capture be a preferable approach to migrate functionality?

Answer: Change data capture (CDC) is ideal in cases where direct interception of calls into the monolith is not feasible, allowing for reactions to data changes without modifying

More Free Book



Scan to Download



Listen It

existing code, especially useful in adapting legacy systems.

6.Question

When using a database wrapping service, what primary benefit does it provide during microservice migration?

Answer:A database wrapping service isolates the underlying database complexity by creating an API layer, enabling applications to interact with the database without directly coupling to it, thereby reducing the risk of disruptive changes.

7.Question

Explain how Spotify benefits from using component-oriented UIs in their applications.

Answer:Spotify utilizes a component-oriented UI structure that allows for quick changes and A/B testing of features without requiring full app deployments, as individual components can be updated independently.

8.Question

What is one of the limitations of using views in database management to facilitate microservice migration?

Answer:Views in databases are often read-only and may not

More Free Book



Scan to Download



Listen It

provide the flexibility needed for dynamic schema changes, which can hinder efforts to evolve the underlying data models effectively.

9.Question

Why is it crucial to address ownership when creating database views?

Answer:Clear ownership of database views ensures that the schema remains up-to-date and well-maintained, preventing unintended consequences from changes in the underlying data structure.

10.Question

How can dynamic changes to a mobile application impact user experience compared to a web application?

Answer:Unlike web applications that deliver updates seamlessly, dynamic changes to mobile apps require users to download new versions, which can lead to delays in feature accessibility and a disrupted user experience.

Chapter 5 | Q&A

1.Question

What is the key challenge when transitioning from a

More Free Book



Scan to Download



Listen It

monolithic database to microservices?

Answer: The key challenge in transitioning from a monolithic database to microservices is determining where data should reside. This involves deciding which data to migrate to new microservices and which data should remain in the monolith. As services are decoupled, maintaining data integrity and consistency becomes complex, especially if services need to interact with aggregated data that is still controlled by the monolith.

2.Question

How can microservices effectively manage data they need from the monolithic database?

Answer: Microservices can effectively manage data from the monolithic database by exposing required data through well-defined APIs or service endpoints. This allows them to access necessary information without tightly coupling to the monolith. It's crucial that the interfaces clearly define which data and operations are permissible, enabling the

More Free Book



Scan to Download



Listen It

microservices to interact with the monolith while maintaining their independence.

3.Question

What strategy should be employed when an invoice service needs to pull data that is still owned by the monolith?

Answer:In such cases, the invoice service should call back to the monolith via a defined service interface to access the required data. While this may seem like an extra step, it is fundamental to preserving the integrity of the microservices architecture long-term, preventing dependencies that could lead to complications in future changes.

4.Question

What approach can be taken when the new microservice requires data ownership transfer from the monolith?

Answer:When transferring data ownership to a new microservice, it's vital to extract the relevant data from the monolith into the new service's database. This process involves reconfiguring the monolith to treat the new microservice as the source of truth for that data henceforth.

More Free Book



Scan to Download



Listen It

Migration requires careful consideration of foreign-key constraints and transactional integrity, typically leading to a phased migration to minimize disruption.

5.Question

What are the benefits and considerations of using a tracer write pattern during data migration?

Answer:The tracer write pattern allows for incremental migration of data by writing changes to both the old and new systems. This approach aids in identifying and resolving inconsistencies early, as changes are synchronized between both sources. However, it introduces complexity, as maintaining two sources of truth can lead to eventual inconsistencies. It's crucial to monitor synchronization and establish a reconciliation process to manage data integrity.

6.Question

What is a primary consideration when synchronizing data between monolithic systems and newly created microservices?

Answer:A primary consideration is to understand the acceptable window of inconsistency between the two

More Free Book



Scan to Download



Listen It

systems, which informs how synchronization is implemented. It's essential to have a strategy to maintain consistency when updates occur across both systems to avoid data integrity issues during the transition phase.

7.Question

How can organizations mitigate the challenges of database decompositions as they shift to microservices?

Answer:Organizations can mitigate these challenges by adopting patterns like repository separation per bounded context which helps isolate database concerns based on specific microservice needs. Additionally, considering logical separation of schemas before physical separation allows for easier management of shared data while preventing a tightly coupled database architecture.

8.Question

What is the recommended approach to handle new functionality that requires data storage when still accessing data from the monolith?

Answer:When implementing new functionality in a microservice, it's advisable to create new data storages, such

More Free Book



Scan to Download



Listen It

as tables, within its own schema while still accessing necessary data in the monolith. This keeps new data isolated and allows the microservice to grow independently from the legacy monolith's constraints.

9.Question

What should teams be wary of when splitting code and databases during microservices migration?

Answer: Teams should be cautious not to fail in completing the transition to independent databases after having split the code. Failing to separate databases while having multiple services could lead to long-term operational issues due to shared state and data access patterns, undermining the benefits of microservices.

10.Question

How does eventual consistency play a role in data synchronization during a transition from a monolithic to microservices architecture?

Answer: Eventual consistency is significant in the transitional phase as it describes the temporary state where data between the monolith and microservices may not be synchronized

More Free Book



Scan to Download



Listen It

immediately. Understanding and planning for this eventual consistency window helps design systems that can gracefully handle periods where data may not match across services.

Chapter 6 | Q&A

1.Question

What are the main challenges when splitting a monolithic database schema into microservices?

Answer: The main challenges include loss of referential integrity, increased latency, and handling data consistency across separate services. When data is split between different databases, traditional database transactions that ensure atomicity are no longer applicable. This requires new ways of managing data interactions and updates in a manner that prevents inconsistencies, which can complicate reporting and data management.

2.Question

How can you handle data consistency when a customer's status can be updated by multiple services?

More Free Book



Scan to Download



Listen It

Answer:One effective strategy is to treat the customer status as part of the customer domain model, managing it through a dedicated Customer service. This way, when different services need to update the status, they make service calls to the Customer service to ensure that updates are consistent and centralized, rather than directly manipulating the shared database.

3.Question

What is a potential consequence of breaking foreign-key relationships when moving to microservices?

Answer:A significant consequence is the potential for data inconsistency. Unlike a monolithic system that enforces these relationships through the database, in a microservices architecture, each service might maintain its own version of data, leading to scenarios where one service has a reference to data that another service no longer recognizes or has deleted.

4.Question

What are sagas, and how do they address transaction challenges in a microservices architecture?

More Free Book



Scan to Download



Listen It

Answer:Sagas are a pattern used to manage distributed transactions by breaking down long-lived transactions into a series of smaller, discrete operations. Each operation is executed independently, allowing for compensation in case of failure. This approach avoids the need for centralized locking mechanisms and reduces the risk of long-held locks that can lead to contention in the system.

5.Question

Why might one choose to duplicate static reference data across microservices instead of maintaining a single shared source?

Answer:Duplicating static reference data can be a pragmatic choice when the data changes infrequently, as it reduces coupling between services and allows for more independent operation. If the data does not need to be consistent across all services, maintaining local copies can simplify operations and reduce latency associated with inter-service communication.

6.Question

How can you ensure that a microservice remains resilient

More Free Book



Scan to Download



Listen It

to the failure of other services it depends on?

Answer: Implementing graceful handling of failures, such as returning a specific message when requested data becomes unavailable, can maintain service integrity. Additionally, using event-driven architectures can allow services to communicate important changes, like deletions, thereby ensuring that other services can adjust their state accordingly.

7.Question

What role do events play in managing data consistency across microservices?

Answer: Events can serve to inform other services of changes in state, such as the addition or removal of data. By publishing events whenever significant data changes occur, a service can maintain a local view of the data that is consistent with the overall system, enabling better coordination and error handling.

8.Question

How does the two-phase commit algorithm work, and what are its limitations?

More Free Book



Scan to Download



Listen It

Answer: The two-phase commit (2PC) algorithm allows distributed systems to ensure coordinated transactions across multiple services. In the first phase, a coordinator requests a 'vote' from all participating services. If all agree, it proceeds to the commit phase where changes are applied. However, 2PC can be problematic due to complexities in managing locks and potential inconsistencies if some services fail to respond, leading to a lack of isolation and increased latency.

9.Question

What is a compensating transaction in the context of sagas, and why is it important?

Answer: A compensating transaction is a specific operation designed to undo the effects of a previously committed transaction. This is crucial in sagas because, unlike traditional transactions, not all operations can be rolled back after they have occurred. Compensating transactions ensure that if a process fails, the overall state of the system can still be corrected to reflect appropriate business logic.

10.Question

More Free Book



Scan to Download



Listen It

When deciding how to manage shared static data, what factors should inform your approach?

Answer: Considerations should include the frequency of data changes, the impact of data consistency, the complexity of maintaining a centralized system versus localized copies, and the specific use cases for the data across different services. Evaluating these factors can help determine whether to share data through a service, duplicate it, or hold it in a dedicated schema.

More Free Book



Scan to Download



Listen It

Ad



Scan to Download



Try Bookey App to read 1000+ summary of world best books

Unlock **1000+** Titles, **80+** Topics

New titles added every week

Brand



Leadership & Collaboration



Time Management



Relationship & Communication



Business Strategy



Creativity



Public



Money & Investing



Know Yourself



Positive Psychology

Entrepreneurship



World History



Parent-Child Communication

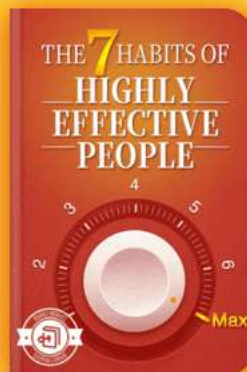
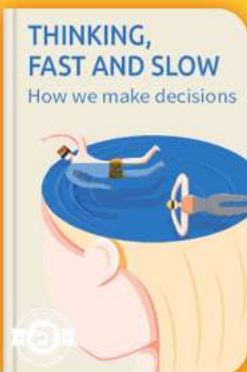


Self-care



Mind & Spirituality

Insights of world best books



Free Trial with Bookey



Chapter 7 | Q&A

1.Question

What is the downside of using orchestrators in a microservices architecture?

Answer:Using orchestrators like the Order

Processor can lead to a higher degree of domain coupling, meaning that services become tightly linked and dependent on one another. It can also lead to centralizing too much logic in the orchestrator, making the services themselves anemic and less autonomous.

2.Question

How can we avoid centralization when orchestrating microservices?

Answer:One way to avoid centralization is to assign different services as orchestrators for different flows. This allows each service to own its local state and behavior while still participating in larger workflows without becoming overly dependent on a single orchestrator.

More Free Book



Scan to Download



Listen It

3.Question

Why are BPM tools often not effective in microservices orchestration?

Answer:BPM tools are usually designed for non-developers to define business processes but often end up being used by developers instead. This can create issues such as poor version control and a lack of testing, making it better for developers to use tools they understand, like coding.

4.Question

What are choreographed sagas and how do they differ from orchestrated sagas?

Answer:Choreographed sagas distribute responsibility across multiple services in an event-driven approach, where services react to events rather than a single orchestrator commanding them. This allows for a more loosely coupled system but can complicate tracking the state of a saga.

5.Question

How can you track the state of a saga in a choreographed approach?

Answer:By using a correlation ID associated with each event

More Free Book



Scan to Download



Listen It

in the saga, you can create a system that collects all related events and presents a view of the saga's state, facilitating better monitoring and potential recovery actions.

6.Question

What are the growing pains organizations face when transitioning to a microservices architecture?

Answer:As organizations adopt microservices, they often encounter issues related to service ownership at scale, breaking changes between services, reporting, monitoring and troubleshooting, as well as increasing complexity in deployments and configurations.

7.Question

How should teams manage ownership of microservices in a growing organization?

Answer:Implementing strong code ownership helps ensure that each service has dedicated owners responsible for changes. This approach not only prevents chaos but also enhances accountability and domain expertise within teams.

8.Question

What common challenge arises with end-to-end testing in

More Free Book



Scan to Download



Listen It

a microservices architecture?

Answer:End-to-end testing can become cumbersome and time-consuming as the number of services increases, leading to long test completion times and increased vulnerability to environmental issues that can cause false negatives.

9.Question

How can organizations improve the developer experience as they scale microservice usage?

Answer:Organizations should implement tools that allow for better management of service interactions, such as stubbing out unnecessary services during local development, and using remote development setups that connect to a more powerful infrastructure.

10.Question

What strategies can teams adopt to handle breaking changes in microservice architectures?

Answer:Teams should aim to minimize breaking changes by maintaining explicit contracts and schemas, using consumer-driven contracts for cross-service dependencies,

More Free Book



Scan to Download



Listen It

providing time for consumers to adapt to changes, and ensuring thorough testing of contract changes.

11.Question

How can organizations mitigate the issue of orphaned services in microservice systems?

Answer:Creating a simple in-house service registry can help track active services, their ownership, and their operational status. This can prevent services from becoming orphaned and ensure that responsibility is clearly defined.

12.Question

What balance should organizations strike between global and local optimization in microservice architectures?

Answer:Organizations must foster communication between teams to align on decisions that have global implications while still allowing local teams to make reversible decisions independently. Cross-team technical groups can facilitate this balance.

13.Question

How can you ensure resilience in a microservice architecture?

More Free Book



Scan to Download



Listen It

Answer:By isolating services, employing patterns like circuit breakers, implementing sensible timeouts, and documenting production issues, organizations can enhance system resilience against failures that become more likely in distributed environments.

More Free Book



Scan to Download



Listen It



Scan to Download



Why Bookey is must have App for Book Lovers



30min Content

The deeper and clearer interpretation we provide, the better grasp of each title you have.



Text and Audio format

Absorb knowledge even in fragmented time.



Quiz

Check whether you have mastered what you just learned.



And more

Multiple Voices & fonts, Mind Map, Quotes, IdeaClips...

Free Trial with Bookey



Monolith to Microservices Quiz and Test

[Check the Correct Answer on Bookey Website](#)

Chapter 1 | Quiz and Test

1. Implementing a new feature like allowing customers to choose a music genre involves coordinated efforts across multiple tiers: Web UI, Backend, and Database.
2. Microservices should share databases to enhance independent deployability and reduce coupling.
3. High cohesion and low coupling are desirable for stability and rapid modification in software design.

Chapter 2 | Quiz and Test

1. Creating autonomous teams allows for improved scalability and efficiency in organizations.
2. Microservices should be implemented in all situations regardless of organizational needs and goals.
3. Focusing solely on reuse as a goal will likely increase complexity and delay delivery.

Chapter 3 | Quiz and Test

More Free Book



Scan to Download



[Listen It](#)

- 1.The organization can reassign responsibilities to handle various areas such as frontend development, backend development, testing, and incident management.
- 2.Self-assessment for skill gaps in transitioning to microservices should be made public to encourage competition among team members.
- 3.To determine if the transition to microservices is successful, only qualitative metrics should be established, disregarding any quantitative measures.

More Free Book



Scan to Download



Listen It

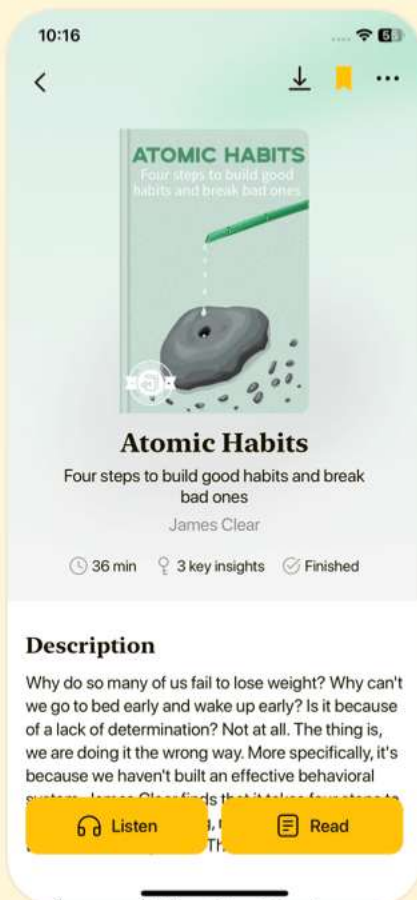


Download Bookey App to enjoy

1000+ Book Summaries with Quizzes

Free Trial Available!

Scan to Download



Chapter 4 | Quiz and Test

- 1.The migration to microservices can be facilitated by breaking the monolith incrementally, often using user interface (UI) decomposition as a guide.
- 2.The shared database strategy is generally encouraged for all types of data during the transition to microservices.
- 3.The 'Parallel Run' pattern allows old and new systems to operate simultaneously, ensuring performance metrics like response times are validated.

Chapter 5 | Quiz and Test

- 1.Data ownership and management during the transition from monolithic architecture to microservices is unimportant and can be disregarded.
- 2.Using a well-defined service interface for accessing data in a monolith can help in preventing tight coupling of systems.
- 3.The best approach is to split the database and code together at the same time during the migration to microservices for a

More Free Book



Scan to Download



Listen It

quicker transition.

Chapter 6 | Quiz and Test

1. Transitioning from a monolithic system to microservices involves splitting data into separate tables for each service.
2. Foreign-key relationships are always useful when data is split across different databases.
3. Sagas are a structure used to handle long-lived transactions across services, requiring distributed locking for coordination.

More Free Book



Scan to Download



Listen It

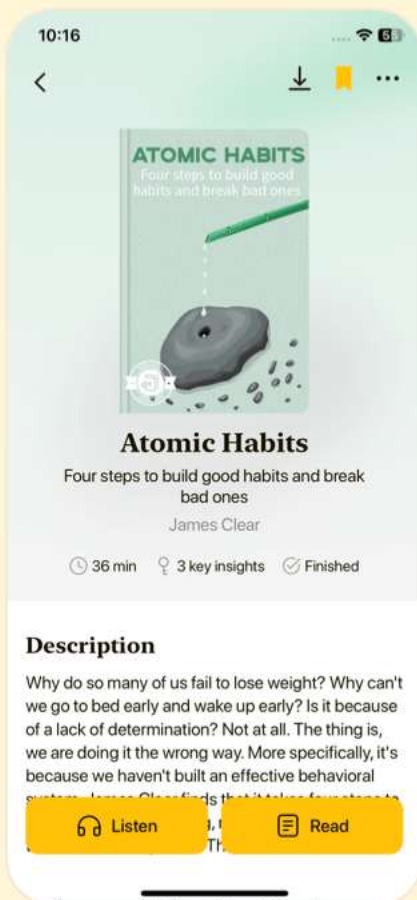


Download Bookey App to enjoy

1000+ Book Summaries with Quizzes

Free Trial Available!

Scan to Download



Chapter 7 | Quiz and Test

- 1.Orchestration can lead to high domain coupling, requiring the orchestrator to know about all associated services.
- 2.Business Process Modeling (BPM) tools are primarily used by non-developers to simplify process definitions without any complications.
- 3.Choreographed sagas simplify tracking the saga's state by eliminating the need for correlation IDs.

More Free Book



Scan to Download



[Listen It](#)



Download Bookey App to enjoy

1000+ Book Summaries with Quizzes

Free Trial Available!

Scan to Download

